



INSTITUTO FEDERAL
Rio Grande do Norte

Desenvolvimento Sistemas Corporativos

Introdução a Spring JPA

Professor Jeferson Queiroga - jeferson.queiroga@ifrn.edu.br



O que é JPA?

- JPA (Java Persistence API) é uma especificação do Java para persistência de dados
- Define um modo padronizado para ORM (Object-Relational Mapping)
- Permite mapear objetos Java para tabelas no banco de dados
- Spring Data JPA é a implementação da JPA no ecossistema Spring



O que o JPA faz

- O JPA permite que você:
 - Crie classes Java que representam tabelas
 - Salve objetos diretamente no banco de dados
 - Faça consultas usando métodos simples
 - Evite SQL repetitivo, usando anotações e abstrações



Dependências no Maven (pom.xml)



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```



Application properties

```
# Configuração do PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/nome_do_banco
spring.datasource.username=seu_usuario
spring.datasource.password=sua_senha
spring.datasource.driver-class-name=org.postgresql.Driver

# Configurações JPA
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```



Criando uma entidade



```
@Entity
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private int idade;
    private String curso;
}
```

@Entity → indica que a classe é uma tabela.

@Id → define a chave primária.

@GeneratedValue → gera o ID automaticamente.



Personalizando a tabela e colunas

```
@Entity
@Table(name = "tb_alunos")
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nome_completo", length = 100, nullable = false)
    private String nome;
}
```

@Table → define o nome da tabela.

@Column → configura o nome, tamanho e obrigatoriedade do campo.



Anotações de Relacionamento

- **@OneToOne**: Relacionamento um para um
- **@OneToMany**: Relacionamento um para muitos
- **@ManyToOne**: Relacionamento muitos para um
- **@ManyToMany**: Relacionamento muitos para muitos



Relacionamento @OneToOne



```
@Entity
public class Endereco {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String cidade;
    private String rua;
}
```



Relacionamento @OneToOne



```
@Entity
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToOne
    private Endereco endereco;
}
```

O JPA criará uma **chave estrangeira** em `Aluno` apontando para `Endereco`.



Relacionamento @ManyToOne

Exemplo: muitos alunos pertencem a **um** curso.



```
@Entity
public class Curso {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
}
```



Relacionamento @ManyToOne

Exemplo: muitos alunos pertencem a **um** curso.

```
@Entity
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @ManyToOne
    private Curso curso;
}
```

O JPA cria uma **coluna curso_id** em **Aluno**.



Relacionamento @OneToMany (parte 1)



```
@Entity
public class Curso {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToMany(mappedBy = "curso")
    private List<Aluno> alunos;
}
```



Relacionamento @OneToMany (parte 1)



```
@Entity
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @ManyToOne
    private Curso curso;
}
```



Entendendo o mappedBy

O atributo **mappedBy** indica quem “possui” o relacionamento. Ou seja, diz ao JPA em qual classe está a chave estrangeira.

Pense assim:

“O lado que tem o **mappedBy** é o lado que **não tem a FK.**”

“O lado que é referenciado no **mappedBy** é o que **tem a FK.**”



Outro exemplo de @ManyToOne e @OneToMany



```
@Entity
public class Professor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @ManyToOne
    private Departamento departamento;
}
```



Outro exemplo de @ManyToOne e @OneToMany



```
@Entity
public class Departamento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToMany(mappedBy = "departamento")
    private List<Professor> professores;
}
```



Relacionamento @ManyToMany



```
@Entity
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @ManyToMany
    private List<Disciplina> disciplinas;
}
```

O JPA cria uma **tabela intermediária** (`aluno_disciplinas`) para armazenar as associações.



Relacionamento @ManyToMany

@JoinTable → permite **personalizar o nome da tabela de junção** e as colunas.

```
@ManyToMany
@JoinTable(
    name = "aluno_disciplina",
    joinColumns = @JoinColumn(name = "aluno_id"),
    inverseJoinColumns = @JoinColumn(name = "disciplina_id")
)
private List<Disciplina> disciplinas;
```

Assim você controla o **nome da tabela**, das colunas e da **chave estrangeira**.



Repository Pattern no Spring Data JPA

- O Repository é um padrão de projeto que atua como uma camada de abstração entre a aplicação e o banco de dados. No contexto do Spring Data JPA, ele simplifica significativamente o acesso aos dados, encapsulando a lógica de persistência e fornecendo uma interface limpa e intuitiva para manipular entidades

```
@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    // Apenas esta definição já fornece todas as operações básicas
}
```

save(): Salvar ou atualizar entidades

findById(): Buscar por ID

findAll(): Listar todos os registros

delete(): Remover registros

count(): Contar registros



Por que o Repository é uma interface?

- No Spring Data JPA, **não precisamos implementar o repositório manualmente**.
- O Spring cria automaticamente uma **classe concreta** em tempo de execução.



```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {  
  
}
```

O **Spring** gera automaticamente a implementação dessa interface.

Você só define a **assinatura dos métodos** (quando quiser métodos extras).

O framework usa **injeção de dependência** para entregar o objeto pronto.



```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {  
  
}
```

Aluno → é a entidade mapeada com **@Entity**.

Long → é o tipo do campo anotado com **@Id**.



Consultas automáticas (Derived Queries)

O **Spring Data JPA** cria consultas automaticamente com base no **nome do método**.
Você não precisa escrever SQL nem JPQL.

Basta seguir o padrão:

findBy + **NomeDoAtributo** + **Operação**



Método no Repository	Significado	SQL Gerado (aproximado)
<code>findByName(String nome)</code>	Busca alunos com nome igual	<code>SELECT * FROM aluno WHERE nome = ?</code>
<code>findByCurso(String curso)</code>	Busca alunos de um curso específico	<code>SELECT * FROM aluno WHERE curso = ?</code>
<code>findByIdadeGreaterThan(int idade)</code>	Maiores que determinada idade	<code>SELECT * FROM aluno WHERE idade > ?</code>
<code>findByIdadeLessThan(int idade)</code>	Menores que determinada idade	<code>SELECT * FROM aluno WHERE idade < ?</code>
<code>findByNameContaining(String texto)</code>	Nome contém um texto	<code>SELECT * FROM aluno WHERE nome LIKE %?%</code>
<code>findByCursoIgnoreCase(String curso)</code>	Ignora maiúsculas/minúsculas	<code>LOWER(curso) = LOWER(?)</code>
<code>findByNotaBetween(double min, double max)</code>	Notas entre dois valores	<code>SELECT * FROM aluno WHERE nota BETWEEN ? AND ?</code>



Combinação de condições

Método	Significado	SQL aproximado
<code>findByCursoAndIdade(String curso, int idade)</code>	Busca por curso E idade	<code>WHERE curso = ? AND idade = ?</code>
<code>findByCursoOrNotaGreaterThan(String curso, double nota)</code>	Curso OU nota acima de valor	<code>WHERE curso = ? OR nota > ?</code>
<code>findByCursoAndNomeContaining(String curso, String texto)</code>	Curso específico e nome contém texto	<code>WHERE curso = ? AND nome LIKE %?%</code>



Ordenação e Limite nos nomes dos métodos

Método	Significado	SQL Gerado
<code>findByCursoOrderByNomeAsc(String curso)</code>	Ordena por nome (A–Z)	<code>ORDER BY nome ASC</code>
<code>findByCursoOrderByIdadeDesc(String curso)</code>	Ordena por idade (Z–A)	<code>ORDER BY idade DESC</code>
<code>findTop3ByOrderByNotaDesc()</code>	Retorna os 3 melhores alunos	<code>LIMIT 3 ORDER BY nota DESC</code>
<code>findFirstByOrderByIdadeAsc()</code>	Retorna o aluno mais novo	<code>ORDER BY idade ASC LIMIT 1</code>

Exemplo completo de **AlunoRepository**



```
@Repository
public interface AlunoRepository extends JpaRepository<Aluno, Long> {

    // Consultas automáticas (Derived Queries)
    List<Aluno> findByCurso(String curso);
    List<Aluno> findByNomeContaining(String nome);
    List<Aluno> findByIdadeGreaterThan(int idade);
    List<Aluno> findByNotaBetween(double min, double max);
    List<Aluno> findByCursoAndIdadeLessThan(String curso, int idade);
    List<Aluno> findByCursoIgnoreCaseOrderByNomeAsc(String curso);

    // Consulta personalizada (JPQL)
    @Query("SELECT a FROM Aluno a WHERE a.curso = :curso AND a.nota >= :notaMin")
    List<Aluno> buscarAprovadosPorCurso(
        @Param("curso") String curso,
        @Param("notaMin") double notaMin);

    // Consulta nativa -- SQL real
    @Query(value = "SELECT * FROM aluno WHERE idade >= :idade", nativeQuery = true)
    List<Aluno> buscarPorIdadeNativa(@Param("idade") int idade);
}
```